

5

Implementing Redux in Our Funbook App

In the previous chapter, we got our hands “dirty” a little bit. I hope you liked building the Funbook app! We managed to build the frontend of a functioning app. Of course, the functionalities we created were limited. A real-world social media app would be much more robust, with many more components and user flows. However, bigger apps bring their own set of problems: handling large datasets, establishing style guides, managing analytics, and many other problems that we don’t want to spend our time on. We are here to talk about different solutions for state management. In the interest of staying focused, I added a few functionalities to our app that were not described in detail in the previous chapter. I added a modal displaying an enlarged version of the images on the **Feed** surface, another modal displaying images added by the users

listed at the top of the **Feed** surface and components and styles for the **Login** surface, and a functioning **Like** button in the modal with the images, connected to the **Profile** surface. You can find the full example app in the **example-app-full** folder on GitHub:

<https://github.com/PacktPublishing/Simplifying-State-Management-in-React-Native/tree/main/example-app-full>.

This app will be the base for all our state management experiments throughout this book. We will start our experiments by looking at the oldest state management library: **Redux**.

In this chapter, we will do the following:

- Go over a brief history of **Redux**
- Install and configure **Redux** in the Funbook app
- Add **Redux** functionalities to the app
- Learn about debugging **Redux**

By the end of this chapter, you should feel comfortable using **Redux**-specific jargon, such as reducer, actions, and store. You should also have a good understanding of what it takes to configure and use **Redux** in a real **React Native** app.

Technical requirements

In order to follow along with this chapter, you will need some knowledge of JavaScript and ReactJS. If you have followed the previous two chapters of this book, you should be able to go forward without any issues.

Feel free to use an IDE of your choice, as React Native does not need any specific functionality. Currently, the most popular IDEs for frontend developers are Microsoft's VSCode, Atom, Sublime Text, and WebStorm.

The code snippets provided in this chapter are here to illustrate what we should be doing with the code. They do not provide the whole picture. To code along easier, please open the GitHub repo in your IDE and look at the files in there. You can either start with the files in the folder named **example-app-full** or **chapter-5**. If you start with **example-app-full**, you will be responsible for implementing the solutions described in this chapter. If you choose to look at **chapter-5**, you will see the entire solution implemented by me.

If you get stuck or lost, you can check the code in the GitHub repo:

<https://github.com/PacktPublishing/Simplifying-State-Management-in-React-Native/tree/main/chapter-5>.

What is Redux? A brief history

We went over a brief history of **React** in *[Chapter 1](#)*, *What are React and React Native?*. If you skipped that chapter, or simply don't remember, don't worry. All you need to know is that **ReactJS** was published in 2013 and it opened doors to creating beautiful single-page applications. **ReactJS** was an exciting library to use! A lot of people jumped on the opportunity and started re-writing their websites. As time passed, many developers would discover that creating and maintaining large applications with **ReactJS** became tedious. Don't forget this was happening before the **ReactJS** team introduced hooks and context. Developers had to pass props from parents to nested children, going through multiple levels of irrelevant components. This is called **prop** drilling, as getting to the child component through many ancestors feels like drilling.

In 2015, something very interesting happened: *Dan Abramov* and *Andrew Clark* wrote and published a new open source library called Redux. ReactJS developers were mostly confused at first, given that **Redux** introduced new concepts to the **ReactJS** world. We could start thinking about global states that are accessible from anywhere in the app. In order to change a global state, we would need to use special functions called “ac-

tions” and also use something called “reducers”... This was a lot to take in! Regardless, this new library solved a very real problem, so the only thing to do was to buckle up, watch Dan Abramov’s tutorials, and use this new and amazing tool!

Thanks to Dan Abramov’s efforts to teach, explain, and popularize **Redux**, it became a staple of **ReactJS** development. As years passed, new concepts for managing global states were created, some similar and some very different from **Redux**. Compared to the newer solutions, **Redux** can feel clunky, as it has a large amount of boilerplate code. Even the library author expressed his doubts through Twitter:



Figure 5.1 – Dan Abramov’s tweet saying he does not understand the Redux example code

Around 2016, the maintenance of **Redux** was passed to *Mark Erikson* and *Tim Dorr*. I had the chance to exchange a few messages with Mark Erikson. He explained to me that he’s not getting paid for maintaining **Redux**; he does it in his spare time,

even though it can be very time-consuming. He says himself that he became a **Redux** maintainer by accident, but after reading his excellent blog post on this topic, I would say he became a **Redux** maintainer because of the amazing amount of work he put into **Redux** documentation and the time he spent helping developers who use **Redux**. You can read the full story on his blog (link in the *Further reading* section). Mark added that he likes maintaining **Redux**. He butts heads with developers who are unhappy with the decisions he's making sometimes, but he also receives support from fellow OSS maintainers, as well as conference invites. I asked Mark what he thinks about **Redux**'s place in the current state management libraries landscape. He pointed out there are many resources (NPM statistics, GitHub statistics, etc.) proving that **Redux** is still by far the most widely used state management library with **React** apps. However, as Mark said, **Redux** was heavily over-used from 2016 to 2017. During that time, a lot of developers raised legitimate complaints about the size of **Redux**'s boilerplate. This situation led in turn to a backlash on Twitter, where a lot of people made claims that "**Redux** is dead" because one tool or another "killed it."

"**RTK** and **React-Redux** hooks changed that narrative. If you look at discussions on Reddit and Twitter today, you do see a good number of folks saying how much they love RTK and recommending it," Mark said.

Redux is currently a mature and trusted solution for managing global states in **React** and **React Native** apps. We've looked briefly at its history in this section. It is obvious that it has its shortcomings. To quote Mark Erikson, "This is a useful tool, not meant for every situation, but a very valid choice." It has its fans and haters, but it's worth knowing about – and that's why we're here! Let's go!

Installing and configuring Redux

As with any library that we would like to add to our project, we will start by reading the documentation. The **Redux** documentation has evolved a lot over the years. In 2022, the recommended install includes **Redux Toolkit**.

Redux Toolkit is the recommended official approach to using **Redux**. It contains commonly used packages and dependencies for building **Redux** apps. This toolkit also simplifies a lot of tasks necessary for using **Redux**, such as creating the store or reducers. Any user is free to install and use core **Redux**, but we will use the recommended approach and use **Redux Toolkit**.

WHY NOT JUST REDUX?

The Redux library has evolved a lot since its conception in 2015. Its ecosystem has also grown a lot. The recommended Redux Toolkit is the most practical addition to Redux apps written in 2022, although it is not a necessity.

Let's start by going into the files for the full app, which is placed in the **example-app-full** folder. Feel free to work directly on those files on your computer. You can also fork the repository or copy the files from this folder. These files include everything you need to run a complete app. If you prefer to follow along with the working code, you should look in the **chapter-5** folder. That is where all completed work for this chapter is placed.

Let's get started. Follow these steps:

1. Once you are inside the **app** folder, run the following command:

```
npm install @reduxjs/toolkit
```

We will go ahead and install the complementary packages recommended in the Redux documentation.

2. Let's run the following commands:

```
npm install react-redux
```

```
npm install --save-dev @redux-devtools/core
```

Now that the dependencies are installed, we can take a minute to talk about **Redux** core concepts.

The main concept, and the absolute most important one, is that with **Redux**, we consider the state a plain object. The **Redux** documentation uses a to-do app as an example, but we can go ahead and use our Funbook app.

If we were to represent the state of the logged-in user of the Funbook app with a single object, it may look something like this:

```
{
  userLoggedIn: true,
  userData: {
    id: 3,
    name: "John Doe",
    email: "john@doe.com",
    image: "imageUrl",
    addedImages: [...],
    likedImages: [...],
    numberOfPosts: 35,
  }
}
```

```
    numberOfFollowers: 1552,  
    numberOfFollows: 128,  
    idsOfFollowedUsers: [...],  
    idsOfConversations: [...]  
  },  
}
```

In this example, we are trying to figure out holistically what user data will be necessary for the entire app. This is what is considered the GLOBAL state. We are not going surface to surface; we want to know all the data relevant to the user.

Therefore, in the **userData** object here, you will find data such as the username and email, which will be used on the **Profile** surface, an array of IDs of followed users, which we can use on the **Feed** surface for the list of avatars, and the array of IDs of conversations necessary for the **Conversations** surface.

Of course, not all our app data is directly dependent on the logged-in user. Let's try and imagine the shape of the part of our global state for the modals present on the **Feed** surface. Here's what the state of the modal opened on an image click may look like:

```
{  
  imageModalOpen: true,  
  imageId: 3,  
  authorId: 3,  
  imageUrl: "imageUrl",  
}
```

```
    numberOfLikes: 28,  
    numberOfConversations: 12,  
    numberOfFollows: 128  
  }
```

Going around the app, we may want to consider the shape of the slice of a global state related to the **Conversations** surface. In my opinion, the data shape we fetch from the fake API set up on GitHub Pages fits very well with the shape of the global state:

```
[  
  {  
    "id": 1,  
    "userId": 2,  
    "text": "Hey, how's it going?"  
  },  
  {  
    "id": 2,  
    "userId": 4,  
    "text": "Yo, are you going to the wedding?"  
  },  
  //...
```

Having the global state be the same shape as the API response is generally welcome. In these cases, you, as the frontend developer, will not have to reshape the data or remember what keys are used where and why. In a perfect world, the API responses would always fit the shape of the data necessary to be

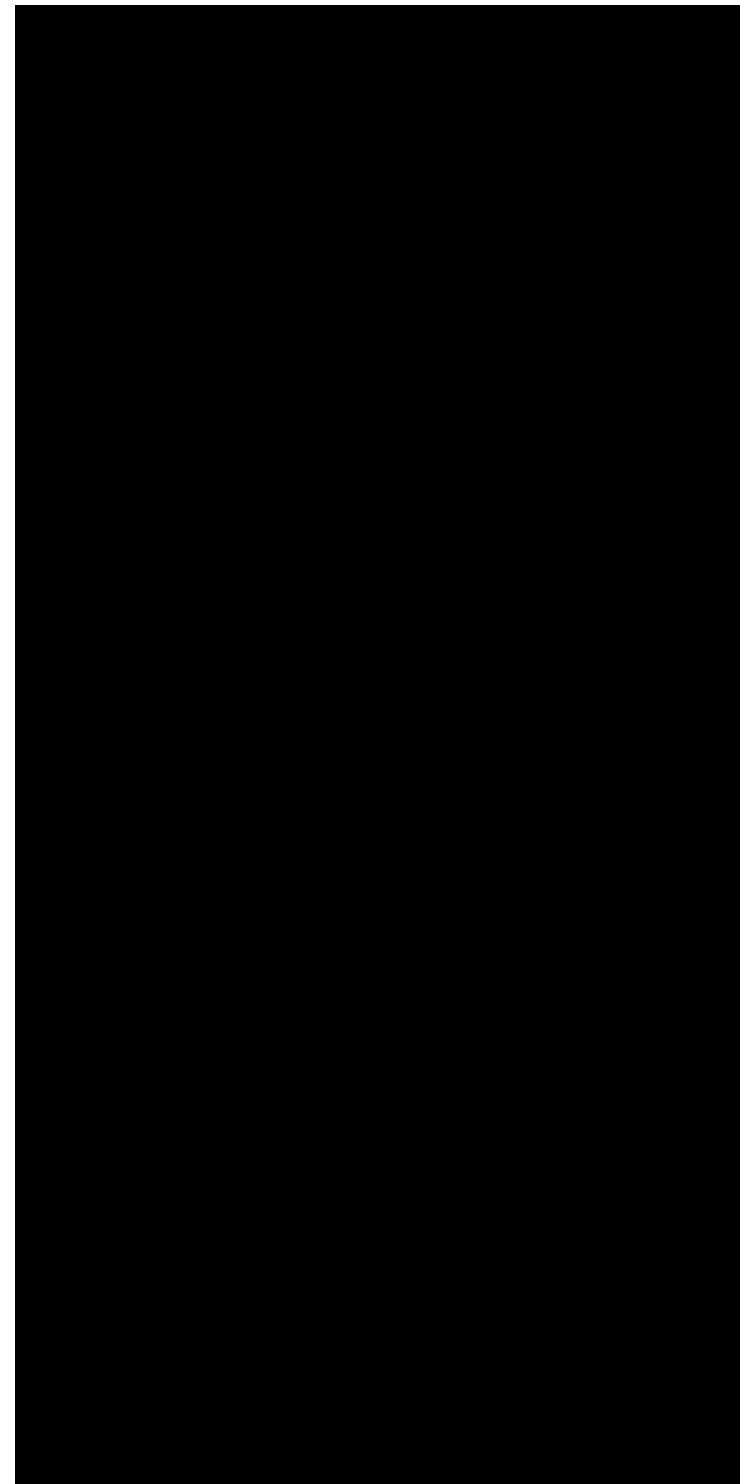
shown on the UI. However, in the real world, that may mean that the frontend would be unnecessarily fetching data that can be shared between surfaces, or fetching unnecessarily big datasets or images.

I feel we are getting the hang of this whole idea of a global state, right? Feel free to try and figure out on your own what other slices of the global state our app may need. Maybe you can sketch out the shape of the global state necessary for the modal displayed when an avatar is pressed – or maybe what exactly is needed for the **Favorited** images surface, and the same data on the **Profile** surface. Come back here when you feel ready to move on to the second **Redux** concept: dispatching actions.

Oh hi! You're back! Great! Let's talk more about **Redux** then!

Dispatching actions

Let's say we've set up the global state – we replaced a lot of unnecessary props and we're happy – but what if we want to change something? What if the user likes an image? What if the user adds a new image or follows another user? We need to tell our state that something has changed. This is when we will dispatch actions. An action is a plain JavaScript object that



describes what is happening. We could dispatch an action that looks like this:

```
{ type: 'LIKE_IMAGE', payload: { Object with data about the liked image } }
```

What now? Has the global state changed magically?

Unfortunately, no. We still need to tell **Redux** to change the state based on this action. The missing piece of this puzzle that ties the actions to the state is called a reducer. Reducer functions are plain **JavaScript** functions that take in the old state and the action and return the new state of the app. Here's what a very simple reducer for liked images may look like:

```
function likedImages(state =[], action) {  
  if (action.type === 'LIKE_IMAGE') {  
    let newLikedImages = state;  
    newLikedImages.push(action.payload);  
    return newLikedImages  
  } else {  
    return state  
  }  
}
```

We are taking in the old state – in this case, the array of liked images. We are then adding the new item and returning the new state. We also get some very elegant error handling in the **else** block, where if there are any problems, the app will return to the old state.

I have described three concepts in this section:

1. **The store** – the single source of truth for the global state
2. **Reducers** – functions that take in the old state and the action, do what you need them to do, and return the new state
3. **Actions** – plain JavaScript objects containing information for the store

These are basically all you need to know to start using **Redux** effectively. If you would like to read more about the concepts and the history of this great library, check out the *Further reading* section where you'll find links to the **Redux** documentation. Now that we know the basics, we are ready to apply this fresh knowledge to a real app.

Adding Redux functionalities to the app

We have installed the **Redux Toolkit** with our friendly package manager in the previous section, but we haven't made any real changes in our app yet. We have, however, thought about the data flows in our app in the previous chapters. The work that we need to do now is going to be very similar. We will start by designing the state structure and actions. When we

have both of those, we will add reducers to tie everything together.

There's a lot of work ahead of us, so let's try to break it down into smaller chunks. We will start by looking at the user state and how we could manage a user's logged-in and logged-out state with a global state in **Redux**. We will then do that same walkthrough for liked images in our app. When we have successfully set up those two pieces of the global state, we will look at how we can combine them and use them in our app. We will then create some actions to handle events in the app. Once we have the state and the actions, we'll take a brief look at how data can be fetched in an app with **Redux**. Finally, we will be ready to get rid of the **React** context we used before for managing the state of our app.

User login state walkthrough

Let's start with the user state. We will create a new file called **store.js** where we will store our initial state slices. We will add this **JavaScript** object to that file:

```
export const user = {  
  userLoggedIn: false,  
  userData: null,  
};
```

When the app is first loaded, we will assume the user is not logged in and there is no user data.

Now, we need to think of an action that will be dispatched when the user is logging in. It should look like this:

```
{type: 'LOGIN', payload: userData}
```

The last part is the reducer. Let's create a new folder for our reducers, called... well, **reducers**. Inside this folder, we will create our reducer file, which should look like this:

```
// reducers/user.js
import { user } from "../store";
export const login = (state=user, action) => {
  if (action.type === 'LOGIN') {
    return {
      ...state,
      user: {
        userLoggedIn: true,
        user: action.payload,
      },
    }
  } else {
    return state
  }
}
```

We are importing our user object as the initial state and then we're adding a switch that will listen to specific actions. Let's listen to the '**LOGIN**' action.

But wait – what if our user would like to sign out? We need another action specifically for this:

```
{ type: 'LOGOUT' }
```

I didn't add any action payload in this case, because we will not be passing any actual data. We only want to wipe the data and we will do that in the reducer. We could add another '**if**' statement to the reducer, but big **if-else** statements become difficult to read and reason about. In the case of reducers, it's a good idea to use the **switch** statement, since we're effectively switching between different states of the app. Here's what our reducer will look like:

```
export const login = (state=user, action) => {  
  switch (action.type) {  
    case "LOGIN": {  
      return {  
        ...state,  
        user: {  
          userLoggedIn: true,  
          user: action.payload,  
        },  
      };  
    }  
  }  
}
```

```
    }  
    case "LOGOUT": {  
      return {  
        ...state,  
        user: {  
          userLoggedIn: false,  
          user: null,  
        },  
      };  
    }  
    default:  
      return state;  
  }  
}
```

OK – now when a user logs in, we will set the global state of the app to reflect that, right? Almost! We still need to find the right place in our code where we will dispatch this action, and that place is the login button on the **Login** surface – but our **Login** surface is shown based on the local state of the main component! That means there’s still a little bit more work that we need to do before we will see the magic of Redux. Don’t worry though, it will be worth it!

IMPORTANT INFORMATION

*If you do have any doubts about all this extra work we seem to be doing, I invite you, my dear reader, to read the **React Navigation** documentation on authentication flows:*

<https://reactnavigation.org/docs/auth-flow/>. In this documentation, you will find that you should not manually navigate when conditionally rendering screens. You would also need to set up the context and preferably use the **useReducer** hook from **ReactJS**. If **useReducer** sounds familiar at this point, that is because it's a **ReactJS** hook with identical functionality to the **Redux** reducers. I hope by now you are starting to feel convinced that using a state management library such as **Redux** is a great solution for React Native apps.

You may wonder why we used a spread operator with the state and then changed the value of **userLoggedIn**. Theoretically, it would be easier to just change the value in the state, no? Not in **Redux**. **Redux** is very adamant about the reducers NOT being able to modify the current state. Reducers can only copy the state and make changes to the copied values. This is important so that our code is predictable. If many reducers changed the same slice of state, who's to say what would be the result?

IMMUTABILITY

This is a very fancy word, isn't it? It means that something is not capable of change, or that it should not be changed. In the case of JavaScript apps, immutable data management can increase performance and make programming and debugging easier. Redux reducers take in the old state and the action and return a

new state object; they should never apply changes to the “old” state object.

If you are curious about the key concepts of **Redux**, I invite you again to the *Further reading* section, where you will find a link to a free course on [Egghead.io](https://egghead.io), created by the author of **Redux**, *Dan Abramov*.

Using Redux for liked images

Our global state is rather poor so far. Keeping the user data in the global state is great but we can surely do more with this great tool. How about liking posts? The reducer for liking posts will look like this:

```
export const likedImages = (state = [], action) => {
  if (action.type === "LIKE_IMAGE") {
    let newLikedImages = state;
    newLikedImages.push(action.payload);
    return newLikedImages;
  } else {
    return state;
  }
};
```

And what if the user decided to unlike a post? Let's add an action and a reducer for this scenario:


```
{ type: 'UNLIKE_IMAGE', payload: { Object with data about the unliked image } }
```

Now, let's adjust our reducer. Since we have multiple actions in a single reducer, we will use a **switch** statement again:

```
// ./reducers/likedImages.js
export const likedImagesReducer = (state = [], action) => {
  switch (action.type) {
    case "LIKE_IMAGE": {
      const newLikedImage = action.payload;
      return [...state, newLikedImage];
    }
    case "UNLIKE_IMAGE": {
      const stateWithoutLikedImage = state.filter(
        (item) => item !== action.payload
      );
      return stateWithoutLikedImage;
    }
    default: {
      throw new Error(`Unhandled action type: ${action.type}`);
    }
  }
};
```

Combining various pieces of global state

We have two reducers, each one meant to manage two different actions. What we need to do now is create a store that will

represent the global state of the Funbook app and pass actions into reducers. We could use the **createStore** function from core **Redux**, but that would require adding more boilerplate files and functions, and it is not the recommended approach for modern **Redux**. The recommended approach is using **Redux Toolkit**, which we will do right now. **Redux Toolkit** offers a special **configureStore** function, which will do a lot of heavy lifting for us. All we need to do is add this function:

```
// ./store.js
import { configureStore } from "@reduxjs/toolkit";
import usersReducer from "./reducers/users";
import likedImagesReducer from "./reducers/likedImages";
export const store = configureStore({
  reducer: {
    user: usersReducer,
    likedImages: likedImagesReducer,
  },
});
```

The **configureStore** function combined our two reducers for us, creating a root reducer required by **Redux**. This single root reducer is required to achieve a single source of truth in the app. This function also adds some useful middleware functionalities, which will check for common mistakes and expose our code for easier debugging.

We created the global state, and we configured it with the reducers thanks to **Redux Toolkit**. Now, we need to tell our Funbook app to use this state. In order to do so, we will use a **<Provider>** component wrapper provided (no pun intended) by the **Redux** library. If you paid attention while we were setting up the app without any exterior libraries, you will have noticed that the **React** context also uses **<Provider>** components. The naming convention is not an accident. Both **<Provider>** components serve the same purpose and **React** context uses a lot of the same high-level logic as **Redux**.

Let's import the necessary elements into our main app file, **App.js**:

```
import { store } from "../store";  
import { Provider } from "react-redux";
```

And let's wrap our app in the Redux **<Provider>**:

```
export default function App() {  
  //...  
  return (  
    <SafeAreaProvider>  
      <Provider store={store}>  
  //...
```

This looks familiar, doesn't it? **Redux's <Provider>** shares a lot of similarities with **React's** context. I cannot give you any links

to official blog posts from the Meta team where React maintainers officially explain this. I can, however, give you my personal opinion that the React team saw the solution that **Redux** was bringing to large **React** apps and thought that some of its principles were worth importing into the **React** repository itself. There are other state management solutions out there, obviously. If there weren't, I wouldn't be able to write this book! Regardless, **Redux** holds a special place in the React ecosystem.

After this short break, we will dive back into our code! We have our store and **Provider** set up. We also have two reducers ready: for user data and liked images data. Let's start with replacing the liked images. We'll go into the **surfaces** folder, where we will find the Favorited surface. This, in turn, will lead us to the component named **ListOfFavorites**, which displays data from the Favorited context.

We will remove this context and use **Redux** data. We will start by importing a **useSelector** hook from Redux, and then we will fetch the actual data from **Redux** using this hook:

```
// src/components/ListOfFavorites
import { useSelector } from "react-redux";
export const ListOfFavorites = ({ navigation }) => {
```

```
const { likedImages } = useSelector((state) =>
//...
```

```
state.likedImages);
```

Do you have our app running on your phone or in the simulator? I hope you do because then you will notice something just went very wrong!

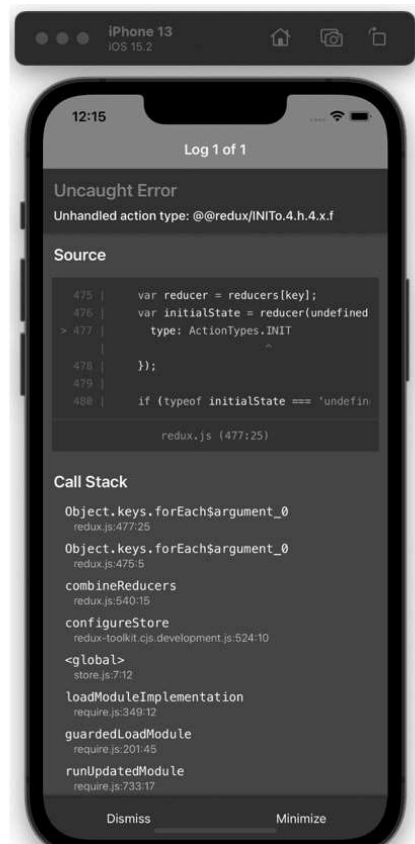


Figure 5.2 – iPhone simulator screenshot with a Redux error

Unhandled action type...? I think I've seen this somewhere...

Oh yes! That's the default in our **switch** statement in the **likedImages** reducer! This is not really the default that we

want, so let's go ahead and change it so that it returns the initial state by default:

```
//reducers/likedImages.js
export const likedImagesReducer = (state = [], action) => {
  switch (action.type) {
    //...
    default: {
      return state;
    }
  }
};
```

The app loads correctly – we're back in business! We are passing the initial state as the default value to the **likedImages** reducer, which means we are passing an empty array – but we want to fetch image data. We did this before in the Context Providers using **fetch.FavoritedContextProvided** used React's **useReducer** hook along with an **init_likes** action dispatched when the images were fetched successfully. When it comes to **Redux**, we do not add functions inside **Provider**. We will create a fetching function inside an action, and then we will dispatch that action when the Favorited surface is rendered. This is a simplistic solution for a simple app. If you are working on a bigger app, you would probably need to concern yourself with caching, avoiding duplicate requests or a cache lifetime. In that case, you should look into a tool provided by

Redux Toolkit, called RTK Query, which simplifies data fetching and caching in Redux apps.

A FULL TOOLBELT

It may start to feel overwhelming learning about so many tools at once. We started with Redux, continued with Redux Toolkit, and now we're adding RTK Query. Don't worry too much at this point about libraries and tool names. We're here to learn how to effectively write an app with a state managed by Redux, and we're following the documentation and best practices to do so. Once you're familiar with the suggested solution, feel free to look around the Redux ecosystem and find the approach that you like the most. There are no wrong answers when it comes to what you like and don't like!

Taking advantage of Redux Toolkit for creating actions

Our reducer is very limited so far. We can't use it directly to fetch data, because as the rules of reducers state, reducers cannot be used to do any asynchronous logic. If we were writing our app sometime around 2018 or 2019, we would probably create a separate **actions** file, manually configure **Redux** middleware functions to manage asynchronous API calls, and finally proceed to write the fetching actions. Luckily, in 2022, we

can take advantage of Redux Toolkit, which comes bundled with all the necessary helper functions and a utility called **createSlice**. A “slice” in **Redux** lingo is a collection of reducers and actions for a single feature in your app. Let’s convert our **likedImages** reducer into a **Redux** Toolkit slice:

```
//reducers/likedImages.js
import { createSlice } from "@reduxjs/toolkit";
export const likedImagesSlice = createSlice({
  name: "likedImages",
  initialState: [],
  reducers: {
    likeImage: (state) => {
      const newLikedImage = action.payload;
      return [...state, newLikedImage];
    },
    unlikeImage: (state, action) => {
      const stateWithoutLikedImage = state.filter(
        (item) => item !== action.payload
      );
      return stateWithoutLikedImage;
    },
  },
});
export const { init, likeImage, unlikeImage } = likedImagesSlice.actions;
export default likedImagesSlice.reducer;
```

Fetching data

Since **Redux** was conceived as a state management tool, it did not come ready to manage fetching data out of the box – but, again, we are using **Redux Toolkit**, which is bundled with the necessary middleware that will let our Redux store digest fetched data. We will use the **createAsyncThunk** function from **Redux Toolkit**.

WHAT'S A THUNK?

*A thunk is a special sort of function that's returned by another function. This name is not related to **Redux** itself.*

Here's what our fetching thunk will look like:

```
import { createAsyncThunk } from "@reduxjs/toolkit";
import { requestBase } from "../src/utils/constants";
export const fetchLikedImages = createAsyncThunk(
  "likedImages/initLikedImages",
  async () => {
    const response = await fetch(requestBase + "/john_doe/
    return await response.json();
  }
);
```

Now, we need to tell our **Redux** slice about this function. We will use the **extraReducers** function provided by **Redux Toolkit** to keep our reducer clean and readable:

```
likedImages.json");
```

```
// reducers/likedImages.js
import { createSlice } from "@reduxjs/toolkit";
import { fetchLikedImages } from "../asyncFetches";
export const likedImagesSlice = createSlice({
  name: "likedImages",
  initialState: {
    likedImages: [],
    loading: true,
  },
  reducers: {
    //...
  },
  extraReducers: (builder) => {
    builder.addCase(fetchLikedImages.pending, (state) => {
      state.loading = true;
    });
    builder.addCase(fetchLikedImages.fulfilled, (state, action) => {
      state.likedImages = action.payload;
      state.loading = false;
    });
    builder.addCase(fetchLikedImages.rejected, (state) => {
      state.loading = false;
    });
  },
});
```

Now that we have a pretty elegant way to manage fetching, including a pending state and rejected state, let's actually fetch our data. We should not fetch it in the **ListOfFavorited** component, because we need to have the image data available as

soon as the entire app is rendered. We should fetch the images in the parent component, **Home**:

```
//src/surfaces/Home
import { fetchLikedImages } from "../../asyncFetches";
import { useDispatch, useEffect } from "react-redux";
// ...
export const Home = () => {
  const dispatch = useDispatch();
  useEffect(() => {
    dispatch(fetchLikedImages());
  }, []);
};
```

This way, the liked images data will be fetched when the app is rendered and the user is on the **Feed** screen. Once the image data is fetched, we can read it from our global state in the **ListOfFavorites** component:

```
//src/components/ListOfFavorites
import { useSelector, useDispatch } from "react-redux";
export const ListOfFavorites = ({ navigation }) => {
  const { likedImages } = useSelector((state) => state.likedImages);
  const dispatch = useDispatch();
  const [imageList, setImageList] = useState([]);
  useEffect(() => {
    const reversedImages = [...likedImages].reverse();
    setImageList(reversedImages);
  }, [likedImages]);
  if (!imageList) {
```

```
    return <AppLoading />;
  }
  //...
  <FlatList
    data={imageList}
    renderItem={renderItem}
    keyExtractor={(item) => item.itemId}
  //...
```

You may have noticed how the fetched data is passed to the state hook:

```
const reversedImages = [...likedImages].reverse();
```

We are using the ES6 spread operator in order to apply the **reverse()** function to a copy of the **likedImages** array. This is because the **likedImages** array is read-only and we cannot operate directly on it.

Replacing the context

Take a moment to look at what you have accomplished. You effectively replaced the Favorited context with Redux! The last thing we need to do is to replace the actions when an image is liked or not and then we'll be ready to do some cleanup!

Let's go into the **ImageDetailsModal** surface and replace context-related code with Redux code:

```
//src/surfaces/ImageDetailsModal
import { likeImage, unlikeImage } from "../../reducers/ likedImages";
import { useDispatch, useSelector } from "react-redux";
export const ImageDetailsModal = ({ navigation, route }) => {
  const { likedImages } = useSelector((state) => state.likedImages);
  const [isCurrentImageLiked, setIsCurrentImageLiked] = useState(false);
  const dispatch = useDispatch();
  useEffect(() => {
    const checkIfLiked =
      likedImages?.filter(
        (favoritedImg) => favoritedImg.itemId ===
      ).length > 0;
    setIsCurrentImageLiked(checkIfLiked);
  }, [likedImages]);
}
```

The last thing we need to change is the function called when the **Like** button is clicked on:

```
<Pressable
  onPress={() => {
    if (isCurrentImageLiked) {
      dispatch(unlikeImage(route.params.imageItem));
    } else {
      dispatch(likeImage(route.params.imageItem));
    }
  }}
/>
```

And we're done with applying **Redux** to the liked images! We can remove the **Favorited** context **Provider**.

Our app consists of functional components only, so we can use **Redux** hooks. If we had class components, we would have to wrap them with special functions called **mapStateToProps** and **mapDispatchToProps**. Modern **React** apps can be built without class components though – as you can see in the Funbook app.

In this section, you learned how to create a Redux store for the user state and liked images. We added reducers for both pieces of the store, as well as actions. We took advantage of a few utilities provided by Redux Toolkit to make our lives easier. We pulled it all together and were finally able to remove a little bit of React's context. Replacing all other pieces of context with **Redux** is a very good exercise to get the hang of this state management library. If you prefer to just take a look at what it would look like, check out the book repo and the folder: <https://github.com/PacktPublishing/Simplifying-State-Management-in-React-Native/tree/main/chapter-5-complete>.

We will now take a look at handling problems and debugging issues that may arise while using Redux.

Debugging

Our Funbook app is quite simple so far. However, when working with bigger apps you will notice that the state becomes more and more complicated with every added feature. Sometimes, features have overlapping states or complex actions, responsible for many things happening across the app. In order to hunt down bugs related to complex state changes, we can use a dedicated debugger. Configuring developer tools in a bare **Redux** app takes a couple of steps, but we're using **Redux Toolkit**! And it comes to the rescue yet again. **Redux Toolkit** is preconfigured to work with the **Redux** DevTools extension, which runs in the browser. Since we are working on a React Native app, we will need to use another tool, called **React Native** Debugger. Mac users can install it using the Homebrew tool:

```
brew install react-native-debugger
```

If you're not using a Mac computer, you will find a prebuilt binary of this app on their installation instructions page: <https://github.com/jhen0409/react-native-debugger>.

Once the remote Debugger is installed, you can run it by typing the following command into your Terminal:

```
open "rndebugger://set-debugger-loc?host=localhost&port=8081"
```

Since we are using **Expo**, there are a few changes we need to make to actually be able to debug our app. So far, the **React Native** Debugger tool with the default config has not found our app:

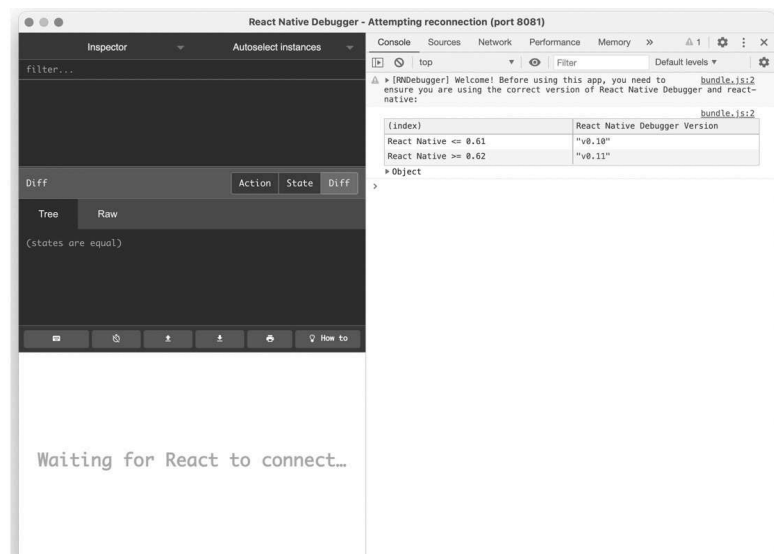


Figure 5.3 – React Native Debugger after installation

We need to tell **React Native** Debugger to look for the right port, which, in the case of **Expo**-managed apps, is **19000**. You will probably need to stop the debugger and the app, then run the following command to open React Native Debugger on the right port:

```
open "rndebugger://set-debugger-loc?host=localhost&port=19000"
```


Finally, restart the app by stopping the server in the Terminal and rerunning it as follows:

```
expo start
```

React Native Debugger is a very useful tool, not only for debugging Redux but also for inspecting all sorts of bugs in **React Native** apps.

In this section, we went over installing and using the **React Native** Debugger tool. I encourage you to look around this very useful tool, inspect the app, and maybe add some bad code to see what an error in this tool may look like.

Summary

We've come a long way on our journey through the state management ecosystem. In this chapter, we talked about what is considered the most common solution for state management in **React** apps – **Redux**. This library has gone through many changes itself. Using it in 2022 is quite different from how it was in 2016 thanks to **Redux Toolkit**, **which** we learned about. We talked about the **Redux** store, reducers, and actions. We've also implemented Redux for liked images in our Funbook app. We're now ready to compare this library to its descendant:

MobX. In the next chapter, we will start by taking a brief look at the history and the high-level ideas behind **MobX**. We will then take **example-app-full** as our starting point and try to replace the **LikedImages** context as we did with **Redux**.

Further reading

- <https://redux.js.org/introduction/why-rtk-is-redux-today>
– Why use Redux Toolkit?
- <https://redux.js.org/introduction/core-concepts> – Redux core concepts.
- <https://blog.isquaredsoftware.com/2016/09/how-i-got-here-my-journey-into-the-world-of-redux-and-open-source/> – Mark Erikson’s blog on how he became a Redux maintainer.
- <https://blog.isquaredsoftware.com/2018/03/redux-not-dead-yet/> – Redux is not dead.
- <https://egghead.io/courses/fundamentals-of-redux-course-from-dan-abramov-bd5cc867> – Egghead tutorial by Dan Abramov.
- <https://stackoverflow.com/a/34582848/8798164> – Stack Overflow answer about state mutations.
- <https://redux.js.org/tutorials/essentials/part-2-app-structure#rules-of-reducers> – Rules of Reducers.

- <https://daveceddia.com/what-is-a-thunk/> – what is a thunk?

